

OpenMP - podstawy: pętla for

Name: Robert Mesek

Date: 10 kwietnia 2026

Zadanie

1. Działający program zdefiniowany w zadaniu na zajęciach (równoległa wersja generowania danych wejściowych do tablicy, dla późniejszego sortowania liczb).
2. Porównanie liczbowe (wykres lub tabelka) przyśpieszenia i czasu wykonania Państwa programu dla różnych: (1) ustawień klauzuli `schedule` (proszę wybrać minimum 5 różnych ustawień: rodzaj `schedule`, parametr `chunk`), (2) wielkości problemu, mierzonego wielkością zaalokowanej tablicy (powinna uwzględniać maksymalne możliwości sprzętu), przy właściwym doborze generatora.
3. Wnioski dotyczące wpływu parametrów klauzuli `schedule` na przyśpieszenie obliczeń dla tego problemu.

Rozwiązanie

1. Jako strukturę danych reprezentującą wektor liczb wykorzystamy typową jednowymiarową tablicę typu `double` z języka C alokowaną na stacku z wykorzystaniem `malloc(array_size * sizeof(double))`. Porównamy dwa generatory liczb pseudolosowych: systemową `rand_r()` (linux.die.net/man/3/rand_r) oraz algorytm `xorshift32`.

```
uint32_t xorshift32(uint32_t* state) {
    // algorithm "xor" from p. 4 of Marsaglia, "Xorshift RNGs"
    uint32_t x = *state;
    x ^= x << 13;
    x ^= x >> 17;
    x ^= x << 5;
    return *state = x;
}
```

2. Przetestujemy rozwiązanie sekwencyjne oraz różne parametry rodzaju `schedule`: `static`, `dynamic`, `guided`, `auto`. Rozwiązanie jest testowane na 12 rdzeniowym/24 wątkowym procesorze, ale ograniczymy liczbę wątków do 12 za pomocą zmiennej środowiskowej `export OMP_NUM_THREADS=12`. Jako jeden z testowanych wartości `chunk_size` wybierzemy rozmiar `loop_count/number_of_threads`. Dla pomiarów wykorzystaliśmy 10 powtórzeń.
- Dla małego problemu (`array_size=100`)

`chunk_size=1`

function	avg [ms]	min [ms]	max [ms]	stddev [ms]
sequential_rand_r	0.000442	0.000426	0.000448	0.000007

function	avg [ms]	min [ms]	max [ms]	stddev [ms]
omp_static_rand_r	0.703282	0.670061	0.807380	0.042237
omp_dynamic_rand_r	0.007332	0.006421	0.012085	0.001706
omp_guided_rand_r	0.006092	0.005451	0.007017	0.000442
omp_auto_rand_r	0.002683	0.002247	0.002865	0.000194
sequential_xorshift32	0.000150	0.000138	0.000256	0.000037
omp_static_xorshift32	0.002581	0.002183	0.003099	0.000315
omp_dynamic_xorshift32	0.005728	0.004993	0.006186	0.000382
omp_guided_xorshift32	0.008293	0.005878	0.025192	0.005954
omp_auto_xorshift32	0.002544	0.002215	0.002917	0.000226

chunk_size=8

function	avg [ms]	min [ms]	max [ms]	stddev [ms]
sequential_rand_r	0.000442	0.000425	0.000448	0.000008
omp_static_rand_r	0.751968	0.662619	0.991346	0.103037
omp_dynamic_rand_r	0.004335	0.003770	0.004823	0.000320
omp_guided_rand_r	0.004714	0.004004	0.005185	0.000376
omp_auto_rand_r	0.002626	0.002279	0.003046	0.000245
sequential_xorshift32	0.000165	0.000138	0.000383	0.000077
omp_static_xorshift32	0.002356	0.002012	0.002715	0.000212
omp_dynamic_xorshift32	0.003410	0.002885	0.004014	0.000332
omp_guided_xorshift32	0.004681	0.003705	0.005430	0.000554
omp_auto_xorshift32	0.002381	0.002066	0.002736	0.000222

- Dla średniego problemu (array_size=300_000)

chunk_size=1

function	avg [ms]	min [ms]	max [ms]	stddev [ms]
sequential_rand_r	2.087223	2.015620	2.149873	0.042612
omp_static_rand_r	1.267070	1.184224	1.341964	0.048530
omp_dynamic_rand_r	8.114654	7.383943	8.710724	0.365164
omp_guided_rand_r	0.115671	0.103195	0.129994	0.010451
omp_auto_rand_r	0.153108	0.118942	0.182400	0.019458

function	avg [ms]	min [ms]	max [ms]	stddev [ms]
sequential_xorshift32	0.398994	0.353908	0.428547	0.026086
omp_static_xorshift32	0.423864	0.294975	0.554952	0.077699
omp_dynamic_xorshift32	7.948742	7.157930	8.362885	0.444637
omp_guided_xorshift32	0.062039	0.050213	0.072562	0.006257
omp_auto_xorshift32	0.064589	0.047424	0.085882	0.015835

chunk_size=25_000

function	avg [ms]	min [ms]	max [ms]	stddev [ms]
sequential_rand_r	2.130417	2.057283	2.362228	0.094652
omp_static_rand_r	0.842754	0.784223	0.914409	0.037919
omp_dynamic_rand_r	0.136884	0.099191	0.179387	0.033273
omp_guided_rand_r	0.130905	0.098083	0.187618	0.033522
omp_auto_rand_r	0.136827	0.097530	0.189151	0.032436
sequential_xorshift32	0.390308	0.353398	0.449224	0.044399
omp_static_xorshift32	0.092400	0.090674	0.093985	0.000912
omp_dynamic_xorshift32	0.053726	0.032794	0.086978	0.014516
omp_guided_xorshift32	0.054310	0.038756	0.076512	0.013199
omp_auto_xorshift32	0.051153	0.032091	0.080356	0.016743

chunk_size=50_000

function	avg [ms]	min [ms]	max [ms]	stddev [ms]
sequential_rand_r	2.122774	2.013900	2.277997	0.077224
omp_static_rand_r	0.932459	0.854219	1.022075	0.048403
omp_dynamic_rand_r	0.233951	0.190876	0.347073	0.057119
omp_guided_rand_r	0.240756	0.190610	0.343485	0.062747
omp_auto_rand_r	0.114942	0.097424	0.172370	0.030413
sequential_xorshift32	0.396167	0.353344	0.512214	0.051617
omp_static_xorshift32	0.072039	0.061936	0.086488	0.009445
omp_dynamic_xorshift32	0.080935	0.063885	0.099415	0.014362
omp_guided_xorshift32	0.078375	0.062617	0.102715	0.014769
omp_auto_xorshift32	0.070024	0.032858	0.125576	0.028150

- Dla dużego problemu (`array_size=1_000_000_000`)

`chunk_size=1`

function	avg [ms]	min [ms]	max [ms]	stddev [ms]
sequential_rand_r	8237.324903	7983.863136	9297.205720	380.514526
omp_static_rand_r	1716.228564	1226.480287	2131.051293	313.426102
omp_dynamic_rand_r	25549.254211	25346.258896	25877.426899	166.705162
omp_guided_rand_r	337.812313	334.947014	346.858949	3.810505
omp_auto_rand_r	338.352596	335.053593	342.390069	1.996123
sequential_xorshift32	1199.727879	1184.636103	1249.642264	20.695370
omp_static_xorshift32	1929.623936	1814.224809	2010.973079	69.672220
omp_dynamic_xorshift32	25732.171792	25362.040152	26116.170725	230.667559
omp_guided_xorshift32	287.124163	272.827227	308.813314	10.779193
omp_auto_xorshift32	292.139381	284.157412	321.453948	11.160562

`chunk_size=41_666_666`

function	avg [ms]	min [ms]	max [ms]	stddev [ms]
sequential_rand_r	8415.481211	7807.053839	10403.186492	824.258685
omp_static_rand_r	431.355420	335.707573	994.409656	201.961260
omp_dynamic_rand_r	337.423365	334.777405	341.004355	1.765320
omp_guided_rand_r	395.163301	391.934390	398.238843	2.035364
omp_auto_rand_r	338.712872	334.860388	344.603821	2.733732
sequential_xorshift32	1193.266137	1179.037086	1212.961556	12.228614
omp_static_xorshift32	329.882931	282.606361	631.502008	106.387136
omp_dynamic_xorshift32	302.342151	277.539946	463.118433	56.691876
omp_guided_xorshift32	291.195842	276.146419	338.367345	18.278014
omp_auto_xorshift32	292.340664	279.020034	338.369217	17.192255

`chunk_size=83_333_333`

function	avg [ms]	min [ms]	max [ms]	stddev [ms]
sequential_rand_r	8093.181217	7887.736839	8326.469586	160.348006
omp_static_rand_r	711.812622	334.205901	1735.168439	576.374964
omp_dynamic_rand_r	409.348204	334.532024	934.999079	188.840301

function	avg [ms]	min [ms]	max [ms]	stddev [ms]
omp_guided_rand_r	394.439557	334.232114	908.091583	180.493235
omp_auto_rand_r	360.636151	333.430730	565.272663	71.933819
sequential_xorshift32	1182.602893	1174.265823	1210.079435	10.408489
omp_static_xorshift32	297.007404	288.205638	315.587334	7.947635
omp_dynamic_xorshift32	300.199296	279.677135	326.731428	15.235039
omp_guided_xorshift32	291.042016	274.893820	306.875936	10.795116
omp_auto_xorshift32	293.130166	279.560436	306.985462	9.259098

chunk_size=166_666_666

function	avg [ms]	min [ms]	max [ms]	stddev [ms]
sequential_rand_r	8203.654274	7851.521947	8934.805057	341.670000
omp_static_rand_r	736.934809	659.629189	1163.829280	154.718813
omp_dynamic_rand_r	699.109109	652.326941	1026.398152	115.090039
omp_guided_rand_r	664.451594	656.242071	672.178337	4.927811
omp_auto_rand_r	339.095403	336.290534	341.843598	1.759861
sequential_xorshift32	1185.099684	1176.442556	1199.802408	7.650903
omp_static_xorshift32	310.936504	299.758904	326.968617	8.977056
omp_dynamic_xorshift32	332.169027	293.702048	398.685509	39.650738
omp_guided_xorshift32	310.628835	298.055155	322.599748	7.056400
omp_auto_xorshift32	302.899558	294.481932	319.226946	7.920733

3. Wnioski

- Dla małego rozmiaru problemu, najlepsza jest wersja sekwencyjna ze względu na narzuty zarządzania wątkami.
- Algorytm generowania liczb pseudolosowych `xorshift32` jest znacznie szybszy oraz jego czasy wykonania są bardziej stabilne od systemowego `rand_r`.
- Najgorszy czas jest w przypadku ustawienia `schedule(dynamic, 1)`, gdzie przy dużych pętlach, każdy wątek po wykonaniu jednej krótkiej operacji, przenosi się na kolejną iterację.
- Bardzo dobrą opcją w naszym przypadku okazała się opcja `schedule(auto)`, w której nie podajemy parametru `chunk_size`.